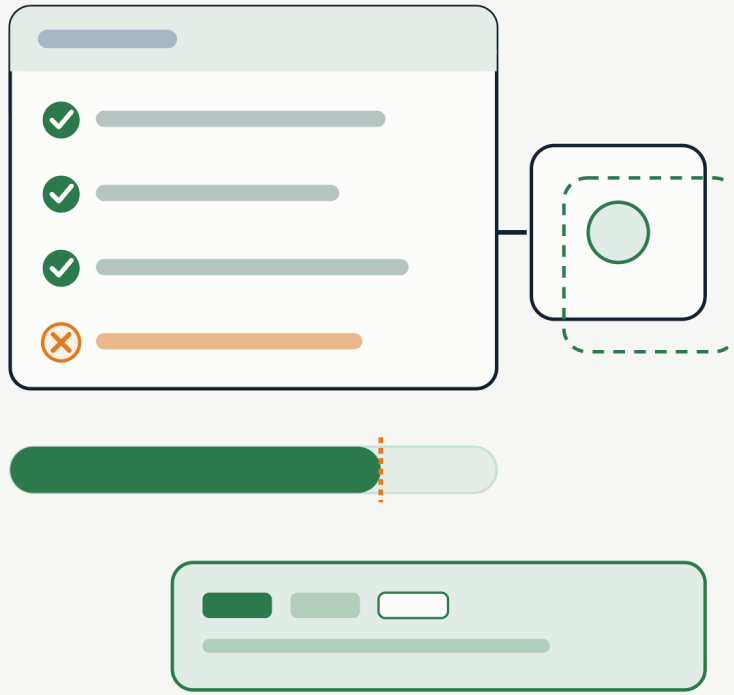
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО
ЕЛЕМЕНТИ РАЗВОЈА СОФТВЕРА

Тестабилни дизајн, NUnit, Moq и покривеност кода

Тестови као доказ пословног понашања и квалитета архитектонских граница.



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРЕГЛЕД

Садржај презентације

01

Структура теста

Тумачење резултата, AAA образац и негативни сценарио.

04–07

02

Test double и Moq

Врсте замена, када користити mock, Setup и Verify, стабилност теста.

08–13

03

Сценарији и покривеност

План тестирања, TestCase, coverage и анализа ризика.

14–21

04

Регресија и КТ1

Репродукција грешке, manual-core-baseline, AI задатак и припрема.

22–28

Увод · пројектовање тестабилног кода (03)

УВОД

Пројектовање тестабилног кода

Раздвајање одговорности, јасни уговори и убризгавање зависности омогућавају контролисано тестирање.

Раздвајање одговорности

мала јединица са једном улогом

+

Јасни уговори

граница коју тест може да замени

+

Dependency injection

зависности стижу споља

=

Изолован тест

изоловано и детерминистички

НАПОМЕНА

Тестови обезбеђују повратну информацију о квалитету архитектонских граница.

01

ПРВИ ДЕО

Структура теста

Како се тест чита, како се пише и шта тачно проверава.

Test Explorer | Arrange, Act, Assert | Назив теста | Негативни сценарио

01 · СТРУКТУРА ТЕСТА

Тумачење резултата у Test Explorer-у

Неуспешан тест усмерава дијагностику, а укупан пролаз потврђује само извршене сценарије.

Test Explorer · ERS.Tests — Прошло 3 — Пало 1

- ✓ Reserve_WhenFree_ReturnsSuccess
- ✓ Reserve_WhenOverlap_ReturnsConflict
- ✓ Reserve_WhenInactive_ReturnsFailure
- ⚠ Cancel_AfterDeadline_ReturnsFailure — **пао**

3 теста су прошла, један захтева анализу. 184 ms

КАКО СЕ ЧИТА РЕЗУЛТАТ

Очекивано
`CancellationWindowExpired`

Добијено
`Success`

Порука показује да ли је проблем у правилу, имплементацији или очекивању теста.

> **Црвени тест прво треба разумети, па тек онда мењати код.**

01 · СТРУКТУРА ТЕСТА

AAA структура теста

Један тест проверава једно јасно понашање.

Arrange
улази

→

Act
операција

→

Assert
исход

→

Diagnose
порука

→

Refactor
сигнал

НАЗИВ ТЕСТА · МЕТОДА_УСЛОВ_ИСХОД

```
Reserve_WhenPeriodOverlaps_ReturnsConflict
```

НАПОМЕНА

Назив теста описује понашање које тест штити.

01 · СТРУКТУРА ТЕСТА

Негативни сценарио из критеријума прихватања

Тест проверава понашање система, а не приватне детаље имплементације.

C# · NUnit и Moq

```
[Test]
public void Reserve_WhenPeriodOverlaps_ReturnsConflict()
{
    // Arrange
    var repository = new Mock<IReservationRepository>();
    repository.Setup(r => r.HasOverlap(It.IsAny<Guid>(),
        It.IsAny<DateTime>(), It.IsAny<DateTime>())).Returns(true);
    var service = new ReservationService(
        repository.Object, new FixedClock(Now));
    // Act
    var result = service.Reserve(
        TestRequests.ValidFutureReservation());
    // Assert
    Assert.That(result.Success, Is.False);
    Assert.That(result.Error, Is.EqualTo("OverlappingReservation"));
}
```

НАПОМЕНЕ

Arrange припрема контролисаног сарадника.

Act извршава једну релевантну операцију.

Assert проверава посматрани исход.

02

ДРУГИ ДЕО

Test double и Moq

Када и како заменити спољну зависност, а да тест остане веран понашању.

- Улога test double-а
- Врсте замена
- Критеријуми за mock
- Setup и Verify
- Стабилност теста

Улога test double објеката

Test double се користи за изолацију конкретне спољне зависности.

ПРЕДМЕТ ТЕСТА

ReservationService

Reservation, Period
стварни објекти, не mock

Мали value object или доменска класа чије понашање је предмет теста најчешће треба да буде стварни објекат.

→ **IReservationRepository**
mock: контролисана вредност

→ **IClock**
FixedClock: детерминизам

→ **HTTP клијент**
stub: унапред задат одговор

НАПОМЕНА

Mock објекти су оправдани на променљивим спољним границама: репозиторијум, HTTP клијент, време и случајност.

Врсте test double објеката

Назив зависи од тога шта замена ради у тесту. Подела потиче из књиге xUnit Test Patterns (Meszaros, 2007).

Врста	Шта ради	Пример у ERS-у
Dummy	само попуњава параметар који се не користи	празан logger
Stub	враћа унапред задат одговор	HasOverlap враћа true
Fake	поједностављена радна имплементација	in-memory repository
Spy	бележи позиве ради касније провере	бројач позива Save
Mock	проверава очекиване интеракције	Verify(Save, Times.Once)

У Moq-у исти Mock<T> објекат служи као stub када користимо Setup, а као mock када користимо Verify.

Критеријуми за употребу mock објеката

Mock објекат контролише променљиву спољну зависност чија имплементација није предмет теста.

Ситуација	Одлука
Repository или HTTP клијент	mock контролише повратну вредност
IClock или генератор ID-ја	mock обезбеђује детерминизам
Мали value object	користити стварни објекат
Доменска класа која се тестира	не mock-овати предмет теста

Moq: Setup и Verify

Setup контролише сарадника, а Verify проверава значајну интеракцију.

C# · Moq

```
1  var repository = new Mock<IReservationRepository>();
2  repository.Setup(r => r.FindById(reservationId))
3            .Returns(existingReservation);
4
5  var result = service.Cancel(reservationId);
6
7  repository.Verify(r => r.Save(existingReservation),
8      Times.Once);
```

НАПОМЕНЕ

> Verify је оправдан када је интеракција део понашања.

> Превише Verify провера прави крхак тест.

> Не проверавати сваки интерни позив без потребе.

Setup шта сарадник враћа · улога stub-а

Verify шта је сарадник примио · улога mock-а

Стабилност теста при рефакторисању

Тест треба да остане стабилан када се мења приватна имплементација без промене понашања.

Тестирати

- видљиви резултат
- стабилан код неуспеха
- значајну интеракцију
- гранично понашање

Не везивати се за

- приватну методу
- редослед непотребних позива
- тренутну структуру петље
- тривијалне setter-е

ТРЕЋИ ДЕО

03

Сценарији и покривеност

Који тестови носе највећу вредност и шта нам извештај о покривености заиста говори.

- Извођење сценарија
- План тестирања
- TestCase
- Coverage
- Линије и гране
- Анализа ризика

03 · СЦЕНАРИЈИ И ПОКРИВЕНОСТ

Извођење тест сценарија из правила

Тестови са највећом вредношћу покривају пословне ризике и граничне услове.

01

Успешан ток

валидни подаци и очекиван резултат

02

Негативни ток

очекивано одбијање захтева

03

Гранични случај

вредност која мења понашање

НАПОМЕНА

Добра пракса: границу правила проверити вредностима са обе њене стране.

План тестирања резервисања

План наводи сценарио, очекивани исход и потребне зависности.

Врста	Пример
Успешан сценарио	опрема је слободна, корисник овлашћен, термин валидан
Негативни сценарио	термин се преклапа са постојећом резервацијом
Негативни сценарио	опрема је неактивна
Гранични сценарио	нови термин почиње тачно када се претходни завршава
Инфраструктурни квар	repository пријављује грешку

Гранични случајеви кроз TestCase

Један параметризован тест проверава сва три случаја правила преклапања.

C# · NUnit · илустрација

```
[TestCase("10:00", "12:00", "11:00", "13:00", true)]   // преклапање
[TestCase("10:00", "12:00", "12:00", "14:00", false)]  // додир на граници
[TestCase("10:00", "12:00", "13:00", "14:00", false)]  // раздвојени термини
public void Overlaps_ReturnsExpected(
    string aFrom, string aTo, string bFrom, string bTo, bool expected)
{
    var a = Period.Parse(aFrom, aTo);
    var b = Period.Parse(bFrom, bTo);
    Assert.That(a.Overlaps(b), Is.EqualTo(expected));
}
```

Сваки TestCase се у извештају приказује као посебан тест.

Гранични ред штити правило полуотвореног интервала.

Period је илустративан тип, назив прилагодити пројекту.

Покретање тестова и анализа покривености

Покривеност кода показује извршене гране и служи као дијагностички сигнал.

terminal · корен solution-а

```
dotnet test

# ако је collector подешен (пакет coverlet.collector)
dotnet test --collect:"XPlat Code Coverage"
```

НАПОМЕНЕ

- Најпре дефинисати пословно важне сценарије.
- Coverage користити као дијагностички сигнал.
- Висок проценат не доказује одсуство грешака.

`dotnet test`
извршава тестове

→

`coverage.cobertura.xml`
сирови подаци collector-а

→

`извештај`
линије и гране по класи

Покривеност линија и покривеност грана

Свака линија може бити извршена, а да једна грана правила никада не буде проверена.

C# · Cancel

```
1   if (r.IsCancelled) return Result.Fail("AlreadyCancelled");
2   r.Cancel();
3   return Result.Ok();
```

▷ Једини тест: отказивање активне резервације.

100%
покривеност линија · 3 од 3

50%
покривеност грана · 1 од 2

НАПОМЕНА
Негативни ток AlreadyCancelled никада није извршен, иако извештај приказује сваку линију као покривену.

Ограничења показатеља покривености

Проценат покривености не замењује анализу пословног ризика и квалитета теста.

> **Питање није „колики је проценат”, већ који непокривени део носи највећи ризик и који тест би дао највише нових информација.**

НАПОМЕНА

Извештај о покривености служи за идентификацију непроверених ризика.

Анализа извештаја о покривености

Извештај служи за идентификацију непроверених грана и ризика.

01 **Условне гране**
да ли су оба исхода проверена

02 **Обрада грешака**
да ли грешка не постаје лажни успех

03 **Критично стање**
новац, дозволе, резервације

04 **Непокривен ризик**
образложити или покрити тестом

ЧЕТВРТИ ДЕО

04

Регресија и КТ1

Тест као заштита од поновљене грешке и стабилна основа пре AI фазе.

Reproduce, Fix, Protect

manual-core-baseline

Захтеви КТ1

AI предлог сценарија

Припрема

Тест за репродукцију грешке

Тест прво репродукује проблем, а затим потврђује исправку и спречава регресију.

01 — црвено

Reproduce

тест показује тренутни bug

→

02 — зелено

Fix

минимална измена понашања

→

03 — трајно

Protect

тест остаје као регресиона заштита

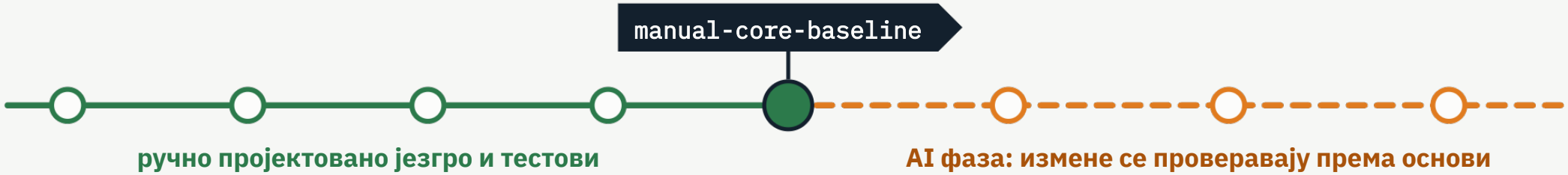
НАПОМЕНА

Тест који је пре исправке пролазио није доказ да је грешка репродукована.

04 · РЕГРЕСИЈА И КТ1

Контролна тачка КТ1

До ове тачке функционално језгро мора бити ручно разумљиво и тестирано.



СТАБИЛНА ВЕРЗИЈА

После КТ1 AI добија већу улогу, али систем већ има структуру и тестове за независну проверу.

КТ1 је у термину Вежбе 4 и носи највише 10 поена.

04 · РЕГРЕСИЈА И КТ1

КТ1: manual-core-baseline

Стабилна верзија језгра служи за проверу каснијих измена уз AI.

01	Кључни use-case-ови имају тестове за успешне и негативне сценарије.
02	Coverage је прегледан и ризична грана је покривена или образложена.
03	Најмање један bug је прво репродукован тестом.
04	Стабилна верзија језгра означена је Git tag-ом manual-core-baseline.
05	Термин КТ1: Вежба 4 · највише 10 поена.

```
git tag manual-core-baseline   ·   git push origin manual-core-baseline
```

Самостални задатак: AI предлог тест сценарија

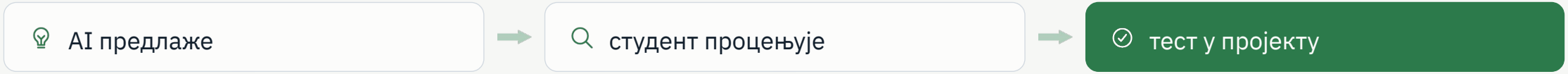
AI може предложити сценарије, а тим бира релевантне тестове и образлаже избор.

prompt · задатак за AI

```
# Задатак за AI
Предложи недостајуће тест сценарије за овај use-case.
Не мењај код.
Образложи пословни ризик сваког предлога.
```

БОДОВАЊЕ

- Први бод: нови смислен гранични тест.
- Други бод: образложење прихватања или одбијања предлога.
- Одговорност остаје код студента.



04 · РЕЗИМЕ

Добре архитектонске границе омогућавају кратке и стабилне тестове усмерене на пословно понашање.

Када су границе система добре, тестови постају краћи, јаснији и ближи пословном понашању које студент треба да одбрани.

правило → сценарио → тест → доказ

Ови резултати су основа за интеграцију и контролисан AI радни ток.

04 · РЕГРЕСИЈА И КТ1

Припрема за интеграцију

Припремити стабилно функционално језгро и тестове за интеграцију модула.

01

Написати план тестирања за један use-case пре тест кода.

02

Додати успешне, негативне и граничне NUnit тестове.

03

Mock користити само на јасним спољним границама.

04

Покренути coverage и анализирати једну ризичну грану.

05

Означити стабилну верзију језгра када је спремна.

Стабилна верзија се означава tag-ом manual-core-baseline за КТ1.